

Animação Computacional de Corpos Rígidos

RELATÓRIO PARCIAL

Programa Unificado de Bolsas de estudo para apoio à formação de estudantes de graduação
(PUB-USP)

João Pedro Neves
(Bolsista)

Afonso Paiva Neto
(Orientador)

São Carlos
2026

RESUMO

O principal objetivo deste projeto de pesquisa é o estudo de problemas de Geometria Computacional aplicados a Animações 3D e à modelagem física, dada a relevância desta área para a Computação Gráfica. O trabalho foca na simulação da dinâmica de corpos rígidos e de fluidos, utilizando algoritmos embasados em leis físicas para reproduzir movimentos realistas em ambiente virtual. Adicionalmente, o projeto visa a capacitação técnica no uso da biblioteca gráfica OpenGL (Open Graphics Library), ferramenta essencial para o desenvolvimento de aplicações gráficas e jogos. A implementação das simulações é realizada utilizando a linguagem de programação Python, explorando sua integração com bibliotecas de computação científica. Os resultados parciais demonstram a viabilidade técnica da modelagem proposta e a eficácia dos algoritmos de integração numérica utilizados.

Palavras-chave: Geometria Computacional. Dinâmica de Corpos Rígidos. Computação Gráfica. OpenGL. Python.

APRESENTAÇÃO

O presente relatório tem por objetivo apresentar as atividades e procedimentos desenvolvidos no período de 01/09/2025 a 27/02/2026, referentes ao projeto de Iniciação Científica vinculado ao Programa Unificado de Bolsas (PUB - USP). As atividades foram realizadas pelo aluno de graduação João Pedro Neves, sob orientação do(a) Prof(a). Dr(a). Afonso Paiva Neto, junto ao Instituto de Ciências Matemáticas e de Computação (ICMC/USP). Durante o período de vigência, o bolsista foi responsável pela implementação computacional dos Capítulos 1, 2, 3 e 9 na íntegra, além da implementação parcial do Capítulo 10 da obra *“Foundations of Physically Based Modeling & Animation”* (Donald House e John Keyser), focando no estudo de modelagem física e animação de sistemas de partículas e corpos rígidos.

INTRODUÇÃO

A Computação Gráfica é a área da computação destinada à geração de imagens em geral — em forma de representação de dados e informação, ou em forma de recriação do mundo real. Ela possui uma infinidade de aplicações para diversas áreas, desde a própria informática, ao produzir interfaces gráficas para software, sistemas operacionais e sites na Internet, quanto para produzir animações e jogos.

Trata-se de uma área que se faz presente em nosso cotidiano e possui meios de trazer avanços tecnológicos relevantes para a sociedade. Hoje em dia, um dos impactos da pesquisa em computação é o aprimoramento de equipamentos médicos utilizados para treinamentos de futuros doutores e doutoras e/ou para diagnósticos clínicos, como softwares que simulam o funcionamento do corpo humano e a fisionomia dos órgãos internos. Outro uso da simulação computacional que causa impacto social positivo é para aquisição de carta de habilitação, que já está sendo utilizada e tem potencial de fornecer ao indivíduo sob treinamento uma melhor experiência com os carros antes das aulas práticas, oferecendo menos risco para o trânsito das cidades. Todas as imagens envolvidas nessas simulações são resultados de estudos em Computação Gráfica.

A computação gráfica também exerce grande impacto sobre o nosso entretenimento e dia-a-dia na sociedade de consumo. Diversos filmes de longa metragem, sejam animações ou live actions, utilizam esses recursos, assim como comerciais de smartphones e carros. Áreas novas com grandes aplicações da computação gráfica também estão surgindo, como a realidade aumentada, realidade virtual, tecnologias interativas, visualização de dados e as inteligências artificiais capazes de gerarem vídeos e imagens personalizadas de acordo com a intenção do usuário.

Nesse contexto, a simulação de corpos rígidos surge como um dos problemas clássicos e fundamentais. Diferente de corpos deformáveis, um corpo rígido é definido como um sistema de partículas onde a distância entre elas permanece constante, independentemente das forças externas aplicadas. No entanto, a implementação computacional desta dinâmica exige a resolução eficiente de equações diferenciais ordinárias (EDOs) e o tratamento preciso de colisões, garantindo que os objetos não se interpenetrem de forma irrealista.

A compreensão das ferramentas técnicas por trás dessas Animações 3D exige a revisão de conceitos de geometria euclidiana, o estudo de algoritmos e estruturas de dados utilizados na resolução de problemas geométricos e os conceitos físicos envolvidos na modelagem destes elementos virtuais. Para o desenvolvimento deste projeto, utiliza-se a linguagem Python em conjunto com a biblioteca OpenGL (*Open Graphics Library*), baseando-se na obra "*House, D.; Keyser, J.; Foundations of physically based modeling and animation*".

Esta experiência propicia ao aluno o contato com aplicações de diversos assuntos vistos em disciplinas de graduação em Matemática, Física e Computação, sendo de grande uso tanto para a carreira acadêmica quanto para a inserção no mercado de trabalho, dada a alta demanda de profissionais qualificados nestas áreas.

OBJETIVOS

O presente projeto visa o estudo e a implementação de conceitos fundamentais de modelagem geométrica e simulação física aplicados à animação de corpos rígidos. Os objetivos dividem-se em:

Objetivo Geral

Capacitar o bolsista no domínio das técnicas de Geometria Computacional e Física Baseada em Modelagem, permitindo a criação, simulação e resolução de problemas de movimento de corpos rígidos em ambientes virtuais 3D.

Objetivos Específicos

- Dominar a utilização da API gráfica **OpenGL** integrada à linguagem **Python 3.13**, estabelecendo o *pipeline* gráfico necessário para a visualização das simulações;
- Estudar e implementar diferentes métodos de **integração numérica**, avaliando sua precisão e estabilidade na simulação de movimentos;
- Investigar e aplicar **estruturas de dados** otimizadas para a aceleração de cálculos físicos e detecção de colisões;

- Aprimorar conhecimentos de **geometria euclidiana e álgebra linear**, realizando a transposição de modelos matemáticos para implementações computacionais eficientes;
- Executar a implementação prática dos algoritmos descritos nos Capítulos 1, 2, 3, 9 e 10 da obra base, consolidando o aprendizado teórico por meio da programação.

MÉTODOS

A metodologia aplicada neste projeto fundamenta-se na transposição de modelos matemáticos e físicos para o ambiente computacional, utilizando uma abordagem de desenvolvimento incremental baseada no livro de House e Keyser (2016).

Ambiente de Desenvolvimento e Ferramentas

Para a implementação das simulações, utiliza-se a linguagem de programação **Python 3.13**. A escolha desta linguagem deve-se à sua vasta gama de bibliotecas científicas e à facilidade de prototipagem de algoritmos complexos. As principais ferramentas de suporte são:

- **PyOpenGL**: *Wrapper* de Python para a API **OpenGL**, utilizado para a renderização gráfica e comunicação direta com a GPU;
- **NumPy**: Biblioteca fundamental para computação científica, empregada na manipulação eficiente de matrizes e vetores essenciais para cálculos de álgebra linear;
- **VS Code**: Ambiente de desenvolvimento integrados utilizados para a escrita e depuração do código.
- **GLFW**: Biblioteca desenvolvida para o uso do OpenGL em multiplataforma, responsável pela criação de contextos, janelas e tratamento de entradas e saídas como mouse e teclado.
- **Matplotlib**: Biblioteca utilizada para elaboração de gráficos diversos utilizados na fabricação deste relatório, desde trajetórias de partículas à análises de desempenho de diferentes algoritmos.

Procedimentos Experimentais e Teóricos

O desenvolvimento do projeto segue as seguintes etapas metodológicas:

1. **Revisão Teórica e Geométrica:** Estudo dirigido dos conceitos de geometria euclidiana, transformações lineares (translação, rotação e escala) e representações de orientação (matrizes e quaterniões).
2. **Implementação de Integradores Numéricos:** Codificação de métodos para a resolução de Equações Diferenciais Ordinárias (ODEs), comparando a estabilidade de métodos como **Euler Explícito** e **Método dos Trapézios** para o cálculo de trajetória e velocidade.
3. **Pipeline Gráfico:** Configuração do ambiente OpenGL, incluindo a definição de câmeras, iluminação básica e a renderização de Primitivas 2D/3D para representação dos corpos rígidos.
4. **Simulação de Dinâmica:** Implementação sistemática dos algoritmos propostos no cronograma, iniciando por sistemas de partículas simples e evoluindo para a dinâmica de corpos rígidos com múltiplos graus de liberdade (Capítulos 9 e 10).
5. **Deteção e Resposta a Colisões:** Desenvolvimento de algoritmos para identificação de contato entre geometrias e aplicação de impulsos físicos para garantir a conservação de momento e a não-interpenetração dos objetos.
6. **Otimização da deteção de Colisões:** Desenvolvimento de estruturas de dados como malhas e QuadTrees para otimização da deteção de colisões de diferentes objetos.

RESULTADOS E DISCUSSÃO

Nesta seção, são apresentados os resultados parciais obtidos durante esse primeiro semestre do período de pesquisa, seguindo uma ordem de complexidade, assim como uma ordem cronológica de acordo com o proposto pelo cronograma de execução do projeto.

Vale ressaltar que diferentes metodologias foram utilizadas em diferentes etapas do desenvolvimento desse projeto a fim de ou criar uma familiarização com diferentes técnicas, ou para alcançar objetivos específicos da maneira mais conveniente.

Inicialização de janela, pipeline gráfico e uso de Primitivas 2D

O primeiro estágio prático do projeto consistiu na configuração do ambiente de desenvolvimento computacional e na validação do *pipeline* gráfico utilizando a linguagem Python integrada à biblioteca **PyOpenGL**.

Como resultado inicial, foi implementado o gerenciamento de janelas e contextos gráficos através da biblioteca **GLFW**. A partir disso, obteve-se sucesso na renderização de primitivas geométricas básicas bidimensionais (como quadrados, triângulos e círculos) na tela.

Este passo foi fundamental para dominar conceitos essenciais de Computação Gráfica, tais como:

- Definição de vértices em Coordenadas Normalizadas de Dispositivo (NDC);
- Utilização da máquina de estados do OpenGL para definição de cores e modos de desenho (ex: `GL_TRIANGLES` e `GL_QUADS`);
- Estruturação do *loop* principal de renderização (*main loop*), responsável por atualizar a tela a cada quadro.
- Capturar eventos de entrada pelo dispositivo, como ações de teclas (Escape para fechar a janela) e de mouse (clique para suspender o movimento de um objeto).

As primeiras tentativas de renderização de imagens contaram com o uso do OpenGL Legacy, ou seja, a forma mais clássica de utilização da API gráfica, que conta com uma simplicidade na execução acompanhada porém de um desempenho abaixo, por delegar todo o trabalho de processamento dos dados de posição quanto de imagem à CPU. Contudo, ao longo do desenvolvimento do projeto, os códigos passaram a ser desenvolvidos com o OpenGL moderno, lançando mão da utilização de shaders e delegando tarefas à GPU. As Figuras 1 e 2 abaixo ilustram o desenho de um quadrado simples colorido na tela, que muda de cor sempre que a tecla “SPACE” é pressionada, demonstrando também a captação de entradas do teclado no escopo do código.

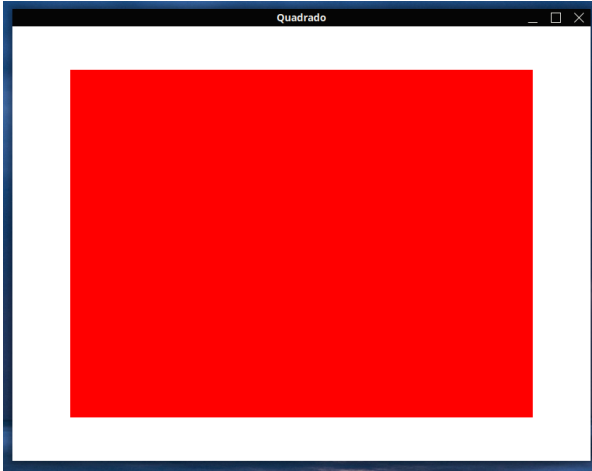


Figura 1. Renderização inicial de uma primitiva geométrica, consolidando o sucesso da inicialização da janela e do loop principal.



Figura 2. Mesma forma geométrica, porém com coloração diferente, consolidando o sucesso da obtenção de dados de entrada, visto que a cor mudou após pressionar a tecla *space*

Transformações geométricas e animação baseada em tempo

Após a validação da renderização estática, o passo seguinte consistiu em introduzir movimento à cena, implementando transformações geométricas afins (neste caso, a translação). Para isso, a arquitetura do projeto foi atualizada para utilizar a biblioteca matemática **GLM** (**PyGLM**) em conjunto com os programas de sombreamento (*Shaders*).

Em vez de recalcular as coordenadas dos vértices na CPU a cada quadro, adotou-se a abordagem moderna do OpenGL: a construção de uma Matriz de Modelo (*Model Matrix*) que é enviada como uma variável uniforme (**uniform**) para o *Vertex Shader*.

Para testar o *pipeline* de animação, aplicou-se uma função trigonométrica à componente horizontal (eixo X) da matriz de translação, definida pela equação:

$$x(t) = A * \sin(\omega t)$$

onde $A = 0.5$ (amplitude do movimento na tela) e ω corresponde ao incremento angular por *frame*. Isso gerou um **Movimento Harmônico Simples (MHS)** contínuo.

Para validar matematicamente a precisão da transformação aplicada pela matriz ao longo do tempo, os dados de posição horizontal foram coletados durante a execução do *loop* principal. O resultado pode ser observado no gráfico da Figura 4, assim como algumas imagens da animação podem ser observadas na Figura 3 abaixo.

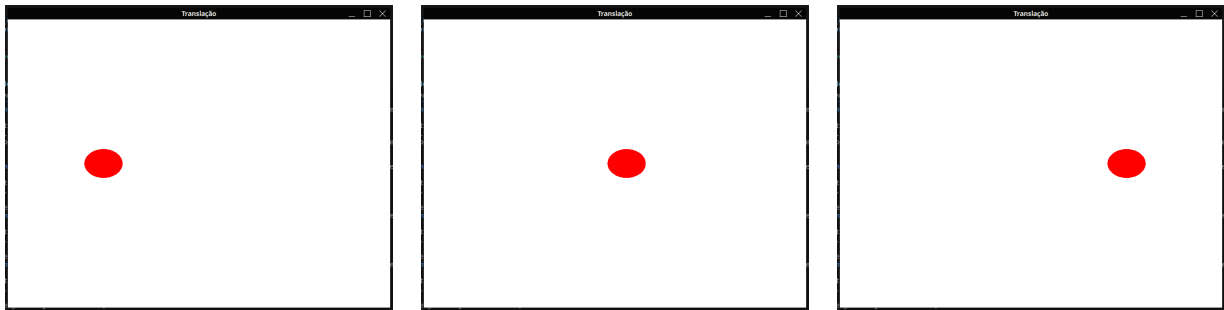


Figura 3. três diferentes frames do movimento de translação periódico obtido.

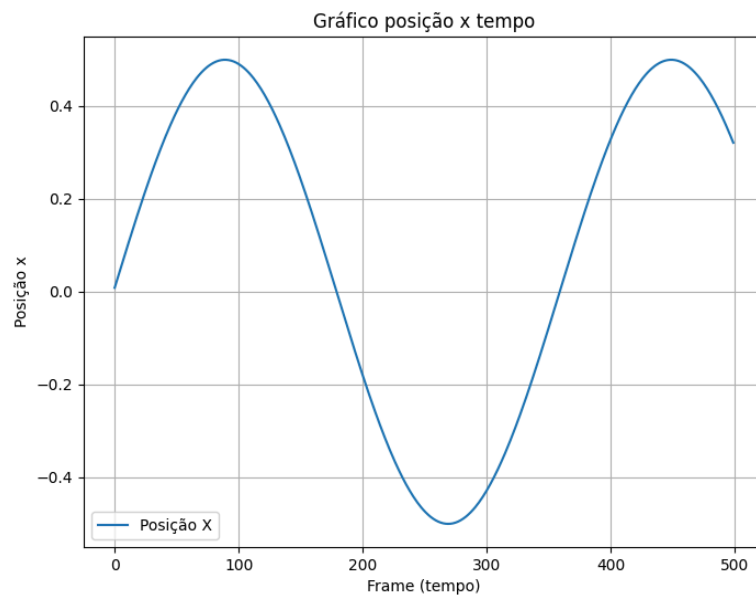


Figura 4. Gráfico da posição em função do tempo do movimento de translação periódico do corpo demonstrado na **Figura 3**.

Integrações e modelos físicos

Uma vez que as tentativas de utilizar o OpenGL moderno, através do uso de shaders e uniforms para atualizar a posição da renderização das imagens, foi-se necessário começar a implementação das noções físicas de animação apresentadas pelo livro [1]. O Capítulo 2 estabelece as bases do que uma animação demanda para ser bem sucedida, a criação de um modelo.

Ao abandonar a alteração direta das coordenadas em favor de atualizar os **vetores de estado** dos objetos, foi possível iniciar a implementação dos algoritmos de integração utilizando os vetores $S(t)$ de estado de uma partícula bidimensional contendo sua posição e sua velocidade, juntamente com os vetores $S'(t)$ contendo as respectivas derivadas, ou seja, velocidade e aceleração, sendo esta última definida pela resultante das forças atuantes no objeto.

$$S(t) = [x \ y \ v_x \ v_y],$$

$$S'(t) = [v_x \ v_y \ a_x \ a_y]$$

Dessa forma, utilizou-se o método de integração simples de Euler para simular eficazmente três modelos distintos de movimento: lançamento horizontal, lançamento horizontal com resistência do ar e por último o mesmo lançamento horizontal sujeito a não apenas a resistência do ar como também a uma força na direção contrária ao eixo x do movimento, a força de arrasto do vento. Cada uma dessas forças foi computada de acordo com as equações descritas no livro [1], a fim de obter as respectivas acelerações para compor o vetor das derivadas. A força resultante está denominada abaixo por F enquanto a força gravitacional por F_g , a força de resistência do ar por F_{air} e a força de arrasto do vento por F_{wind} .

$$F = F_g + F_{air}$$

$$ma = mg - dv$$

$$a = g - \frac{d}{m}v$$

e

$$F = F_g + F_{air} + F_{wind}$$

$$a = g + \frac{d}{m}(v_{wind} - v)$$

onde d equivale a resistência do ar, m a massa do objeto, a e v representam a aceleração resultante e a velocidade do objeto respectivamente, e v_{wind} representa a velocidade do vento. Obtendo assim os seguintes resultados, todos à partir das mesmas condições iniciais de posição e velocidade. As Figuras 5, 7 e 9 representam imagens das animações de um lançamento horizontal, enquanto as Figuras 6, 8 e 10 são os gráficos da trajetória da partícula referente a simulação da respectiva imagem anterior.

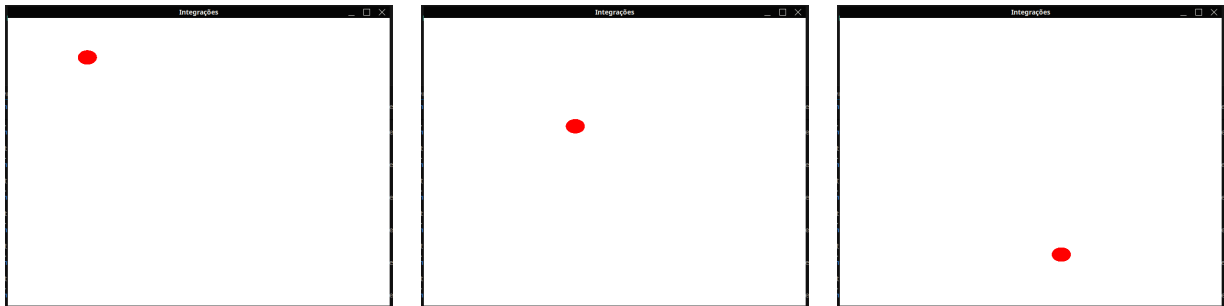


Figura 5. três frames distintos da simulação do movimento de lançamento horizontal sujeito apenas a força da gravidade

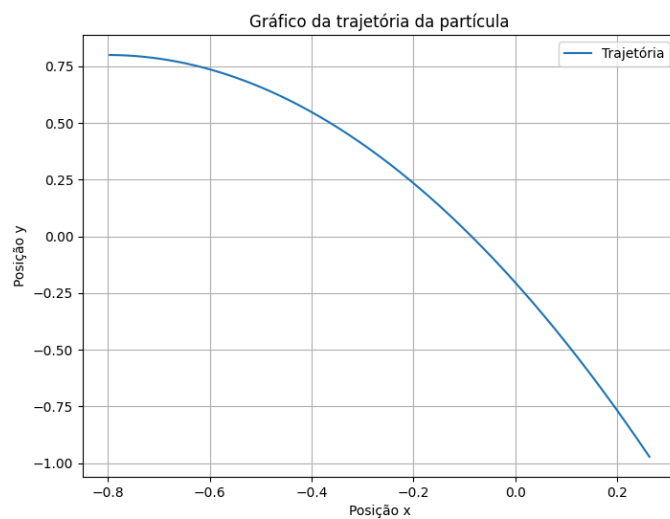


Figura 6. Gráfico da trajetória obtido da simulação do movimento representado na **Figura 5**.

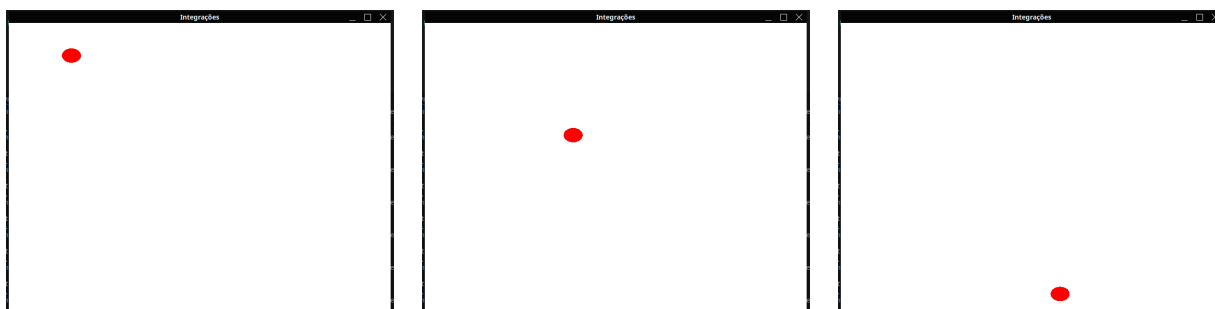


Figura 7. três frames distintos da simulação do movimento de lançamento horizontal sujeito a força da gravidade e a resistência do ar. Nessa simulação, a resistência do ar e a força de arrasto do vento valiam ambas zero.



Figura 8. Gráfico da trajetória obtido da simulação do movimento representado na **Figura 7**. Percebe-se que o alcance total horizontal foi menor graças à resistência do ar que dessa vez foi considerada na resultante das forças.

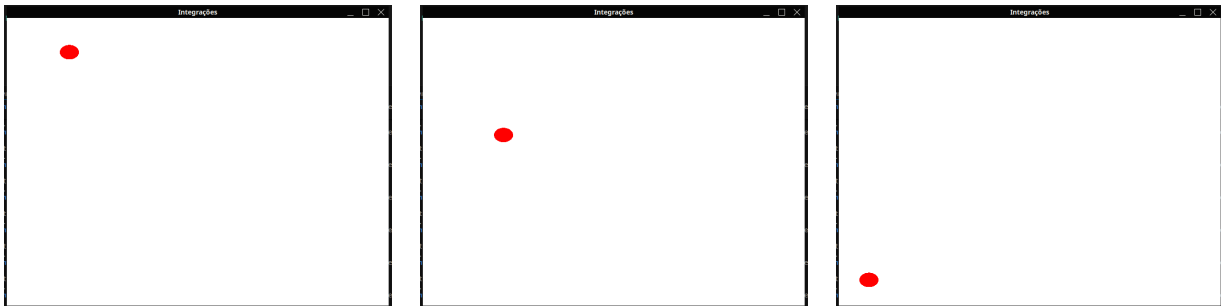


Figura 9. três frames distintos da simulação do movimento de lançamento horizontal sujeito a força da gravidade, resistência do ar e força de vento com velocidade contrária ao movimento

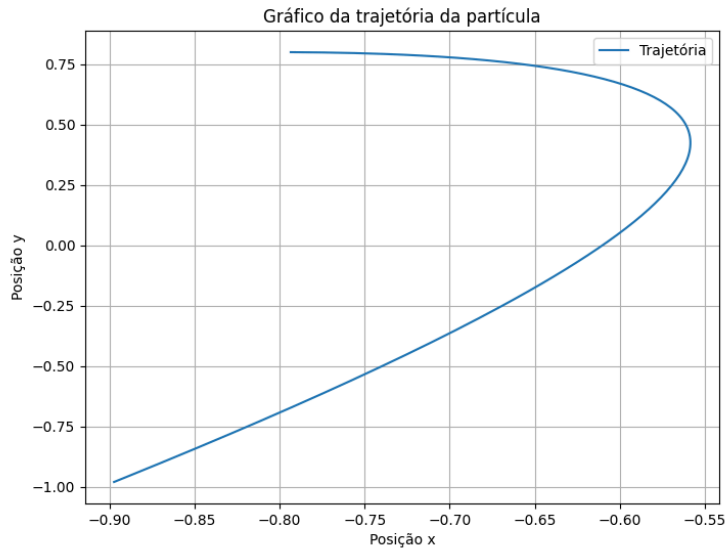


Figura 10. Gráfico da trajetória obtido da simulação do movimento representado na **Figura 9**. A força do vento em sentido contrário à velocidade inicial do objeto conferiu à trajetória um formato completamente distinto dos anteriores.

Em todas as simulações demonstradas, o valor da posição inicial foi $x^{[0]} = [-0.8, 0.8]$, a velocidade inicial $v^{[0]} = [0.4, 0.0]$ e a velocidade do vento $v_{wind} = [-3.0, 0.0]$. As constantes tiveram valor $d = 0.1$ e $m = 1.0$.

Vale ressaltar que as simulações acima foram feitas utilizando o método de integração simples de Euler:

$$v^{[n+1]} = v^{[n]} + a^{[n]}h$$

$$x^{[n+1]} = x^{[n]} + v^{[n]}h$$

Esse método, apesar de menos preciso, foi utilizado apenas com o intuito de propiciar um primeiro contato mais simples com os métodos de integração e de simulação de modelos físicos, tendo em vista que o mesmo posteriormente foi substituído por outros modelos de integração durante a execução do projeto.

Tratamento de colisões

Após concluir as simulações propostas pelo Capítulo 2 [1], partiu-se para o início da formulação dos modelos propostos pelo Capítulo 3, de colisão simples de partículas com planos, à exemplo de uma bola batendo na parede lateral da tela. O material fonte divide primariamente o fenômeno da colisão em duas situações distintas: detecção da colisão e tratamento da colisão.

A detecção da colisão consiste em, a partir do pressuposto que há uma partícula pontual e um plano com uma reta normal, identificar em qual instante de tempo a orientação da partícula muda em relação a essa normal, ou seja, em que momento o objeto deixa de estar na frente do plano para estar do lado de trás, como demonstra a Figura 11.

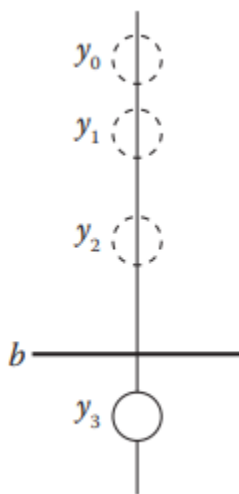


Figura 11. ilustração do momento em que uma partícula deixa de estar de um lado da barreira fixa **b** para estar do outro. Imagem adaptada de [1]

A imagem acima pressupõe que a posição da partícula está demarcada pela coordenada y , e apenas sabendo o sinal de $y - b$, é possível determinar de qual lado da barreira a bola está. Dessa forma, a detecção foi modelada considerando os limites da tela como planos infinitos. Matematicamente, a distância d de um ponto $\mathbf{x}$ a um plano definido por um ponto $\mathbf{P}$ e sua normal $\mathbf{n}$ é dada por $d = (\mathbf{x} - \mathbf{P}) \cdot \mathbf{n}$. Considerando o raio r da partícula, a colisão é detectada quando a distância entre o centro do objeto e o plano é menor ou igual ao raio. No código, isso foi implementado mapeando os limites normalizados da janela OpenGL $[-1, 1]$ nos eixos X e Y e testando a intersecção do estado futuro da partícula com essas fronteiras.

Para determinar o momento exato da colisão, uma abordagem ingênua na simulação causaria a "penetração" da partícula na parede antes que a velocidade fosse invertida. No intuito de evitar esse artefato visual e físico, implementou-se o cálculo do instante exato do impacto através de interpolação linear.

Sabendo a distância da partícula ao plano antes da colisão d_1 e a distância projetada caso ela atravessasse o plano d_2 , calcula-se a fração de tempo $f \in [0, 1]$ em que o impacto ocorreu:

$$f = \frac{d_1}{d_1 - d_2}$$

Na implementação prática, essa interpolação foi realizada relacionando a distância percorrida no eixo da colisão, garantindo que o intervalo de tempo de integração dt fosse fracionado $dt_{colisão} = f * dt$. O restante do tempo ($dt - dt_{colisão}$) é guardado e processado no mesmo quadro, permitindo que a simulação resolva múltiplas colisões em um único *frame* (por exemplo, quando a bola atinge a quina da tela).

Confirmado o impacto, o vetor de velocidade incidente v^- é decomposto em duas componentes relativas à normal n do plano de colisão: a velocidade normal v_n^- e a tangencial v_t^- .

$$v_n^- = (v^- \cdot n)n$$

$$v_t^- = (v^- - v_n^-)$$

A resposta física considera a conservação de momento e a dissipação de energia através de dois fatores: o coeficiente de restituição c_r , que define a elasticidade do choque; e o coeficiente de atrito c_f , que atua sobre a componente paralela à superfície. A nova velocidade de saída v^+ é a soma das componentes resultantes:

$$v^+ = -c_r v_n^- + (1 - c_f) v_t^-$$

Para validar o modelo, os coeficientes foram configurados para uma colisão

perfeitamente elástica $c_r = 1.0$ e sem atrito $c_f = 0.0$, na ausência de gravidade. Os dados da trajetória foram exportados durante a execução e plotados para análise, resultando em rebatimentos perfeitamente refletidos (ângulo de incidência igual ao ângulo de reflexão), comprovando a integridade do modelo vetorial implementado. As Figuras 12, 14 e 16 representam frames de diferentes animações de colisão simples com diferentes valores de c_r e c_f , enquanto as Figuras 13, 15 e 17 contêm os gráficos das trajetórias das partículas demonstradas nas respectivas Figuras anteriores.

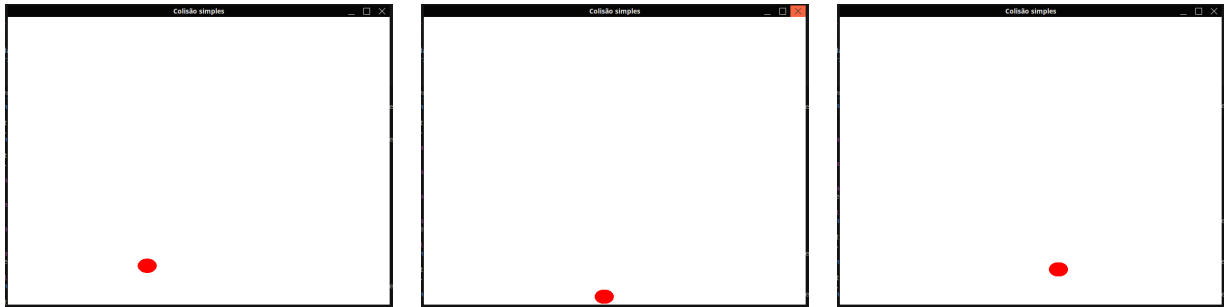


Figura 12. três frames distintos da simulação de uma colisão perfeitamente elástica entre a partícula e a parede inferior da janela de aplicação.

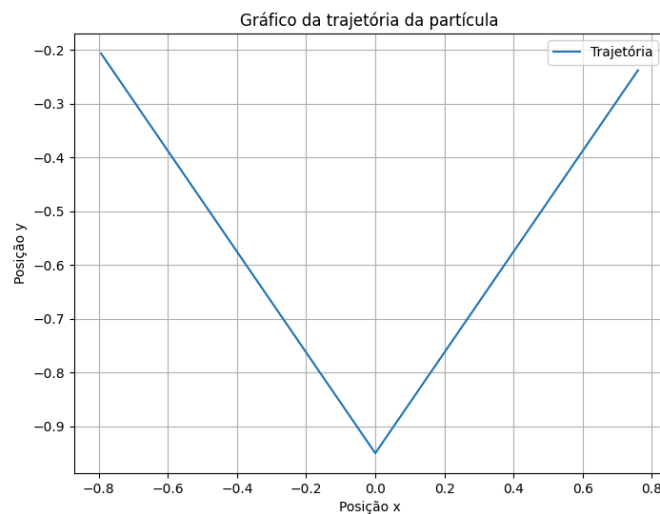


Figura 13. Gráfico da trajetória obtida da simulação da colisão perfeitamente elástica da partícula com a parede inferior da janela de aplicação. Na imagem, nota-se a simetria característica da colisão perfeitamente elástica.

Uma vez que os resultados foram satisfatórios, fez-se o mesmo teste dessa vez com os coeficientes de restituição elástica e de atrito modificados:



Figura 14. Gráfico da trajetória obtida da simulação da colisão com **coeficiente de restituição = 0.5** e **coeficiente de atrito = 0.0**.

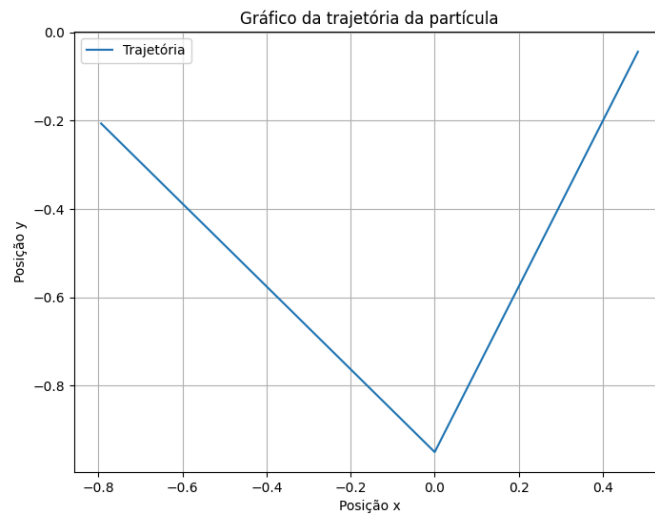


Figura 15. Gráfico da trajetória obtida da simulação da colisão com **coeficiente de restituição = 1.0** e **coeficiente de atrito = 0.5**.



Figura 16. Gráfico da trajetória obtida da simulação da colisão com **coeficiente de restituição = 0.8** e **coeficiente de atrito = 0.8**.

Otimização da Detecção de Colisões com QuadTree

Com o modelo de colisão entre partículas e planos validado, o passo seguinte consistiu

em estender a simulação para cenários com múltiplos objetos simultâneos, o que introduz um novo desafio computacional: a detecção eficiente de colisões entre pares de partículas.

A abordagem ingênua para esse problema — denominada força bruta — consiste em testar todos os pares possíveis de objetos na cena, resultando em complexidade $O(n^2)$. Para n objetos, isso implica $n(n-1)/2$ comparações por frame, tornando-se rapidamente inviável à medida que o número de partículas cresce.

Para contornar essa limitação, foi implementada uma estrutura de dados hierárquica denominada QuadTree, que subdivide recursivamente o espaço bidimensional em quatro quadrantes sempre que o número de objetos em uma região excede uma capacidade predefinida. Cada partícula é alocada na subdivisão correspondente à sua posição, e a busca por candidatos à colisão é restrita apenas às regiões espacialmente próximas, reduzindo a complexidade para $O(n \log n)$.

A detecção de colisões passa a operar consultando, para cada partícula, apenas os vizinhos presentes na mesma região da estrutura hierárquica, em vez de percorrer todos os objetos da cena. A subdivisão espacial é renderizada na tela junto com a simulação, permitindo a visualização direta das regiões geradas pela estrutura a cada frame.

Para validar o ganho de desempenho, o número de partículas na cena foi duplicado em intervalos regulares de tempo, e o número de comparações realizadas por frame foi registrado para ambos os métodos. Os resultados, plotados ao término da simulação, confirmam o comportamento assintótico esperado: enquanto a força bruta apresenta crescimento quadrático no número de comparações, a QuadTree demonstra crescimento significativamente menor, validando a eficácia da estrutura para cenas com alta densidade de objetos, conforme ilustrado nas Figuras 17 e 18 a seguir contendo as simulações sem e com QuadTree respectivamente, e depois na Figura 19 com o gráfico de número de testes de intersecção por quantidade n de objetos para o caso sem a estrutura de dados, e na Figura 20 com o mesmo gráfico mas agora com o uso da estrutura.

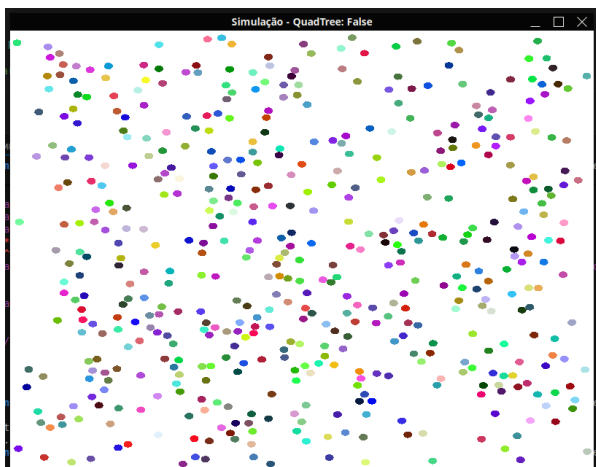


Figura 17. Frame retirado da simulação de colisões entre diferentes partículas com as paredes da janela de aplicação e com elas mesmas. Método de força bruta para determinar a detecção de colisões.

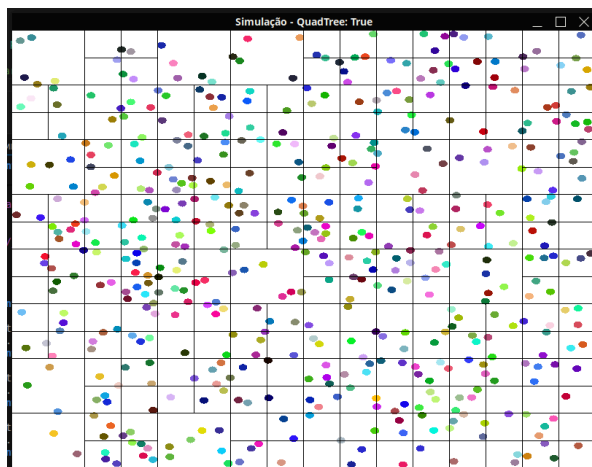


Figura 18. Frame retirado da simulação de colisões entre diferentes partículas com as paredes da janela de aplicação e com elas mesmas. Método de QuadTree representado pelos diferentes contornos quadráticos em preto na imagem.

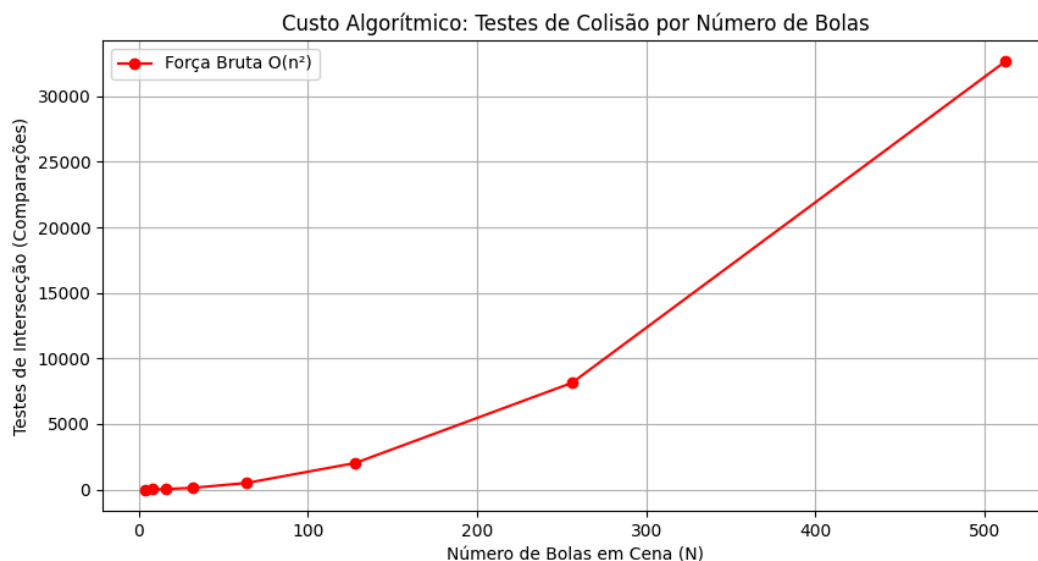


Figura 19. Custo algorítmico da detecção de colisões pelo método de força bruta $O(n^2)$, medido em número de testes de intersecção em função da quantidade de partículas em cena. O crescimento quadrático característico torna o método impraticável já a partir de 500 partículas,

atingindo cerca de 32.000 comparações nesse patamar — número equivalente ao produzido pela QuadTree com mais de 16.000 partículas.

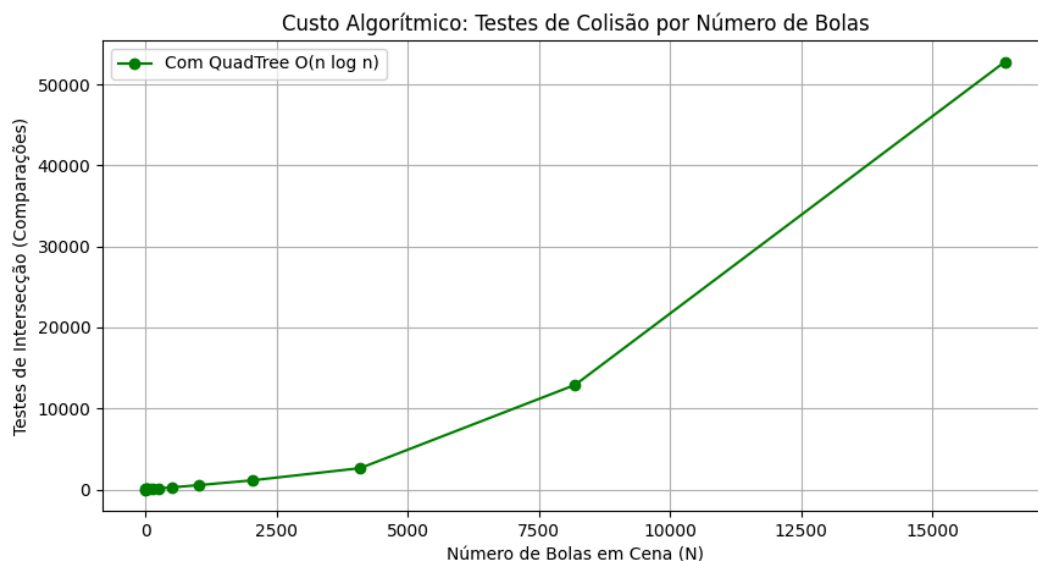


Figura 20. Custo algorítmico da detecção de colisões utilizando a estrutura QuadTree $O(n \log n)$, medido em número de testes de intersecção em função da quantidade de partículas em cena. O crescimento sub quadrático é evidenciado pela capacidade da estrutura de processar até 16.000 partículas com aproximadamente 53.000 comparações por frame.

Simulação de Corpos Rígidos 2D

Com a estrutura de detecção e resposta a colisões entre partículas estabelecida, o projeto avançou para a implementação da dinâmica completa de corpos rígidos bidimensionais, correspondente ao Capítulo 9 da obra base. Diferentemente de uma partícula pontual, um corpo rígido possui extensão geométrica e, portanto, está sujeito não apenas a forças lineares, mas também a torques que produzem rotação em torno de seu centro de massa.

Para representar o estado físico completo do corpo, o vetor de estado foi expandido para contemplar quatro grandezas independentes: a posição do centro de massa, a orientação do corpo, o momento linear e o momento angular. A orientação, em particular, deixa de ser

representada por um simples ângulo escalar e passa a ser descrita por um quaternião unitário — escolha que confere maior estabilidade numérica à integração e evita as singularidades associadas às representações baseadas em ângulos de Euler.

A resistência do corpo à variação de seu estado rotacional é caracterizada pelo tensor de inércia, calculado analiticamente para a geometria retangular utilizada. A partir desse tensor, é possível determinar a velocidade angular instantânea do corpo em função do momento angular acumulado, bem como a contribuição rotacional na velocidade de cada ponto de sua superfície.

A atualização do estado a cada passo de tempo segue o mesmo paradigma de integração adotado nas etapas anteriores: as derivadas de cada grandeza do vetor de estado são calculadas a partir das forças e torques atuantes, e o estado é avançado numericamente pelo método de Euler. A derivada da orientação, expressa no espaço dos quaterniões, é obtida por meio do produto quaterniônico entre o quaternião de velocidade angular e o quaternião de orientação atual, sendo o resultado normalizado a cada passo para preservar a unitariedade da representação.

O tratamento de colisões foi igualmente estendido para o caso de corpos rígidos. A resposta ao impacto com as paredes da janela é calculada com base na teoria de impulsos: para cada vértice do corpo que ultrapassa os limites da cena, computa-se a velocidade local do ponto de contato — composta pela contribuição linear e pela contribuição rotacional — e aplica-se um impulso proporcional ao coeficiente de restituição adotado. Esse impulso atua simultaneamente sobre o momento linear e o momento angular do corpo, garantindo uma resposta fisicamente consistente que preserva a dinâmica rotacional após o impacto.

Adicionalmente, foi implementada a interação via mouse, permitindo ao usuário clicar sobre o corpo durante a simulação para fixá-lo temporariamente na cena. A detecção do clique é realizada transformando a posição do cursor do espaço de tela para o referencial local do corpo, por meio da inversa da matriz de rotação associada ao quaternião de orientação atual, e verificando se o ponto resultante está contido na geometria retangular.

O resultado obtido pode ser observado na Figura 21, que ilustra um frame da simulação com o corpo rígido retangular em movimento de translação e rotação simultâneos dentro da janela de aplicação.

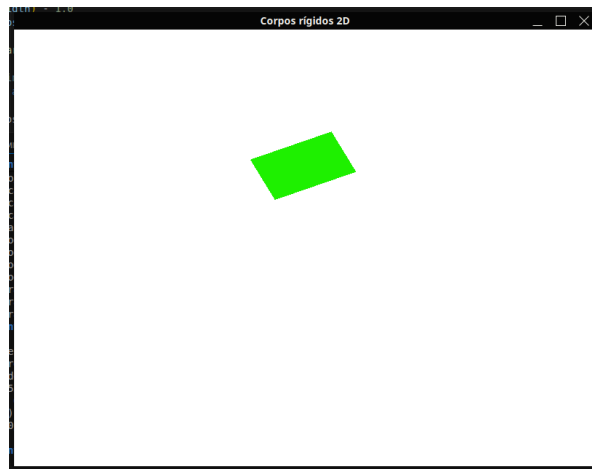


Figura 21. Frame da simulação de corpo rígido retangular 2D, exibindo translação e rotação simultâneas sob a ação das forças modeladas. A orientação inclinada do retângulo evidencia a correta integração do estado rotacional por meio de quatérniões.

Simulação de Corpos Rígidos 3D

A etapa seguinte consistiu na extensão do modelo de corpo rígido para o espaço tridimensional, representando um avanço significativo em relação à simulação bidimensional anterior. A geometria adotada foi a de um cubo, definido por seus oito vértices e seis faces, com a inclusão de normais por vértice para viabilizar o cálculo de iluminação na cena.

O modelo dinâmico segue a mesma estrutura conceitual da implementação 2D: o vetor de estado é composto pela posição do centro de massa, pela orientação representada por um quatérnio unitário, pelo momento linear e pelo momento angular. O tensor de inércia, calculado analiticamente para a geometria cúbica, é utilizado para determinar a velocidade angular instantânea a partir do momento angular acumulado, e sua transformação para o referencial global é realizada a cada passo de integração por meio da matriz de rotação correspondente ao quatérnio de orientação atual.

A principal diferença em relação ao caso bidimensional reside na configuração do pipeline gráfico, que passou a exigir o uso de uma câmera perspectiva posicionada no espaço tridimensional. Para isso, foram introduzidas as matrizes de projeção perspectiva e de visualização, configuradas de modo a posicionar o observador em um ponto elevado e ligeiramente afastado da cena, proporcionando uma visão ampla do ambiente. A iluminação da cena foi configurada com uma fonte de luz pontual posicionada próxima à câmera, permitindo a percepção de profundidade e volume do corpo simulado.

Para contextualizar espacialmente a simulação, foi adicionado um plano de referência em forma de malha quadriculada, representando o chão da cena. Essa malha é renderizada independentemente do corpo rígido, com sua própria matriz de transformação, e serve como referência visual para a percepção do movimento e da orientação do cubo ao longo do tempo.

O resultado obtido pode ser observado na Figura 22, que ilustra um frame da simulação com o cubo em movimento de translação e rotação simultâneos sobre o plano de referência tridimensional.

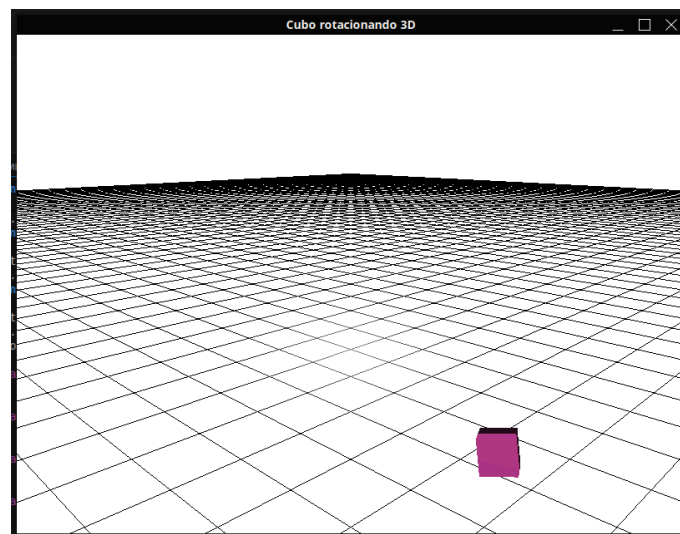


Figura 22. Frame da simulação de corpo rígido cúbico em ambiente tridimensional, exibindo translação e rotação simultâneas integradas por quatérniões. O plano de referência quadriculado ao fundo evidencia a profundidade da cena e a configuração perspectiva da câmera.

Simulações Completas

Ao final da realização das seguintes simulações apresentadas e detalhadas acima, foi elaborada a confecção de um vídeo contendo alguns segundos de todas elas rodando em sequência. Esse mesmo material foi publicado no YouTube para visualização do público geral e pode ser acessado através do link abaixo:

<https://youtu.be/mcOPSjmpYls>

CONCLUSÃO

Os resultados parciais obtidos ao longo deste primeiro semestre de pesquisa demonstram o cumprimento satisfatório das etapas previstas no cronograma de execução do projeto. A implementação progressiva dos módulos — desde a inicialização do pipeline gráfico com OpenGL até a simulação de corpos rígidos em ambiente tridimensional — evidencia a viabilidade técnica da abordagem metodológica adotada e a consolidação gradual dos conceitos teóricos por meio da prática computacional.

A configuração do ambiente de desenvolvimento com PyOpenGL e GLFW estabeleceu a infraestrutura gráfica necessária para a visualização em tempo real das simulações. A transição do OpenGL Legacy para o pipeline moderno, com o uso de programas de sombreado e matrizes de transformação, mostrou-se essencial para a escalabilidade das implementações subsequentes, culminando na configuração de câmera perspectiva e iluminação exigidas pela simulação tridimensional.

A implementação dos integradores numéricos, iniciada pelo método de Euler Explícito, permitiu a simulação de modelos físicos com diferentes configurações de forças externas, cujos resultados foram validados graficamente e apresentaram comportamento consistente com os modelos analíticos esperados. O desenvolvimento do módulo de detecção e resposta a colisões consolidou conceitos fundamentais de decomposição vetorial e conservação de momento, com os testes realizados para diferentes valores de coeficiente de restituição e atrito confirmando a correção do modelo implementado.

A implementação da estrutura hierárquica QuadTree para otimização da detecção de colisões entre múltiplas partículas demonstrou, de forma quantitativa, a importância da escolha

algorítmica no contexto de simulações em tempo real. A comparação direta entre o método de força bruta e a abordagem hierárquica evidenciou que, enquanto o primeiro se torna inviável com poucas centenas de objetos, a QuadTree mantém desempenho satisfatório com dezenas de milhares de partículas em cena.

Por fim, a extensão do modelo dinâmico para corpos rígidos — primeiro no plano bidimensional e posteriormente no espaço tridimensional — representou o avanço mais significativo do período, consolidando o uso de quaterniões para representação de orientação, do tensor de inércia para modelagem da resistência rotacional e da teoria de impulsos para o tratamento fisicamente consistente de colisões. Em conjunto, os Capítulos 1, 2, 3, 9 e parte do Capítulo 10 da obra base foram implementados com êxito, proporcionando ao bolsista uma sólida base técnica em Geometria Computacional, integração numérica de equações diferenciais ordinárias, física baseada em modelagem e estruturas de dados aplicadas à simulação gráfica.

TRABALHOS FUTUROS

Para o segundo semestre de vigência do projeto, as atividades previstas concentram-se na conclusão e no aprofundamento da dinâmica tridimensional de corpos rígidos, com ênfase nos aspectos ainda não implementados. As principais frentes de desenvolvimento incluem:

A implementação do tratamento completo de colisões entre corpos rígidos tridimensionais, estendendo o modelo de impulsos já desenvolvido para o Caso 2D ao Espaço 3D, com detecção de contato entre geometrias volumétricas e resposta física que preserve simultaneamente o momento linear e o momento angular de todos os corpos envolvidos. Além do modelo de impulsos, o **Teorema do Eixo de Separação** é ademais um dos tópicos a ser estudado e implementado no processo de tratamento das colisões entre polígonos tridimensionais.

A investigação e implementação de estruturas de dados hierárquicas adequadas ao espaço tridimensional — como as Bounding Volume Hierarchies (BVH) — para otimização da detecção de colisões em cenas com múltiplos corpos rígidos, dando continuidade ao trabalho de otimização já realizado com a QuadTree no contexto bidimensional.

A substituição do método de Euler Explícito por integradores de maior ordem, como o Método de Runge-Kutta de 4ª ordem (RK4) e o Método dos Trapézios Animação Computacional de Corpos Rígidos com o objetivo de aprimorar a precisão e a estabilidade das simulações de longa duração, especialmente relevante no caso de corpos com dinâmica rotacional intensa.

A consolidação da implementação completa dos Capítulos 9 e 10 da obra de House e Keyser, juntamente com a implementação do Capítulo 11, incorporando não apenas os modelos de forças e torques externos ainda não explorados, como atrito entre superfícies em contato e restrições de movimento entre corpos acoplados, como também modelos de restrições para as simulações de movimentos limitados em um ou mais graus de liberdade.

Ao término dessas etapas, espera-se a obtenção de um sistema funcional e robusto de simulação de corpos rígidos em ambiente tridimensional, consolidando os objetivos centrais do projeto e fornecendo base técnica para eventuais extensões à simulação de fluidos e corpos deformáveis.

REFERÊNCIAS

[1] House, D.; Keyser, J.; Foundations of physically based modeling and animation.; CRC Press, 2017.